AI English Coach Telegram Bot

Architecture, Step-by-Step Build Guide & Deployment Reference Manual

Author / Recipient: Revanth Acharya	**Target Platform:** Telegram (Desktop & Mobile) + Windows
AI Core Model: OpenAI GPT-4o-mini	**API Integration:** Telegram Bot API (Python v21+)
Primary Purpose: 24/7 Grammar & Communication Coach	**Deployment Modes:** Local PC Execution & Cloud (Render)

1. System Overview & Objective

This project establishes a direct bridge between the **Telegram Bot API** and the **OpenAI API** to deploy a personalized, 24/7 AI communication coach. Unlike generic chat bots, this assistant is specifically prompted to analyze every message you send, identify grammatical and vocabulary inaccuracies, provide brief educational explanations, and keep dialogue engaging with natural follow-up questions.

Important Note on OpenAI Billing: A standard ChatGPT Plus web subscription does not include API usage. The OpenAI API operates on a separate developer platform (platform.openai.com) with a pay-as-you-go credit balance. The gpt-4o-mini model used here is exceptionally cost-effective, typically costing less than $0.001 per conversation.

2. Prerequisites & Credentials Setup

Step 2.1: Telegram Bot Token via @BotFather

1. Open Telegram on mobile or desktop and search for the official **@BotFather**.
2. Send /newbot to start the bot creation wizard.
3. Provide a friendly display name (e.g., *Revanth English Coach*).
4. Choose a unique username ending with 'bot' (e.g., revanth_english_coach_bot).
5. Copy and securely store the generated HTTP API bot token (e.g., 8628427747:AAG...).

Step 2.2: OpenAI API Secret Key Setup

1. Log into the OpenAI Developer Platform at https://platform.openai.com/api-keys.
2. Navigate to **API Keys** and click **Create new secret key**.
3. Copy the secret key (starts with sk-proj-...). Store it in a safe, private location.
4. Under *Settings* > *Billing*, add a payment method or top up a small credit balance (e.g. $5).

Step 2.3: Python Environment & Dependency Installation

Open Windows PowerShell or Command Prompt in your project directory and execute:

```
pip install openai python-telegram-bot
```

3. Complete Python Implementation (telegram_ai_bot.py)

Below is the complete, production-ready Python script integrating the asynchronous OpenAI client with Telegram long-polling:

```
import os
from openai import AsyncOpenAI
from telegram import Update
from telegram.ext import (
    Application, CommandHandler, MessageHandler, ContextTypes, filters
)

# Initialize OpenAI client from environment variable
client = AsyncOpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

# Strict system persona enforcing communicative corrections and engagement
SYSTEM_PROMPT = (
    "You are an expert personal English communication coach. "
    "Follow these strict pedagogical rules: "
    "1. Answer user questions clearly and honestly. "
    "2. Gently highlight any grammar, spelling, or vocabulary mistakes in user input. "
    "3. Briefly explain why it was incorrect and provide a natural, native alternative. "
    "4. Conclude with an engaging follow-up question to keep conversation flowing. "
    "5. Do not agree with grammatically incorrect phrasing."
)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    welcome_text = (
        "Hello Revanth! I am your personal AI English Coach.\n\n"
        "Send me any message in English. I will reply while checking your "
        "grammar, vocabulary, and phrasing to help you become fluent!"
    )
    await update.message.reply_text(welcome_text)
```

```
async def chat(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_message = update.message.text
    # Indicate typing state in Telegram while AI computes
    await context.bot.send_chat_action(chat_id=update.effective_chat.id, action="typing")

    try:
        response = await client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_message}
            ],
            temperature=0.7,
            max_tokens=600
        )
        reply = response.choices[0].message.content
        await update.message.reply_text(reply[:4000])
    except Exception as e:
        await update.message.reply_text(f"Error connecting to OpenAI: {e}")

def main():
    token = os.environ.get("TELEGRAM_BOT_TOKEN")
    if not token:
        print("ERROR: TELEGRAM_BOT_TOKEN environment variable is not configured.")
        return

    # Build Telegram bot application
    app = Application.builder().token(token).build()

    # Register command and message dispatchers
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, chat))

    print("==================================================")
    print(" AI English Coach Telegram Bot is live & polling! ")
    print("==================================================")
    app.run_polling()

if __name__ == "__main__":
    main()
```

4. Execution & Windows PowerShell Setup

Set your environment variables in PowerShell and launch the script:

```
# 1. Set your secret API tokens in PowerShell
$env:OPENAI_API_KEY="sk-proj-YOUR_ACTUAL_OPENAI_API_KEY"
$env:TELEGRAM_BOT_TOKEN="YOUR_ACTUAL_TELEGRAM_BOT_TOKEN"

# 2. Run the bot script
python telegram_ai_bot.py
```

5. Strategic Roadmap: Advanced Extensions

Once your base bot is operating, you can progressively enhance it into a full personal language laboratory:

- **Multi-Turn Conversation Memory:** The starter script treats each message individually. By maintaining a sliding window of previous message turns (or saving session logs to a lightweight SQLite database), your coach can track recurring mistakes across sessions and refer back to earlier discussions.
- **Voice Notes & Pronunciation Coaching:** Telegram allows users to record and send voice memos. By downloading the audio and transcribing it via the OpenAI Whisper API, your bot can analyze spoken speech, transcribe it, check pronunciation patterns, and reply with audio synthesized via Edge-TTS or OpenAI TTS.
- **Structured Visual Feedback Cards:** Instead of plain paragraphs, configure the coach to return formatted markdown cards: **Corrected Sentence**, **Grammar Explanation**, **Vocabulary Alternatives**, and **Discussion Prompt**.
- **Continuous 24/7 Cloud Hosting:** Deploy the application to cloud platforms such as Render.com or Railway.app so that your coach is continuously available on your smartphone around the clock without needing your laptop powered on.

Executive Summary: By combining Telegram's instantaneous mobile accessibility with OpenAI's conversational intelligence, you have a private, zero-judgment language tutor always in your pocket. Start with text coaching today, and expand into voice synthesis as your fluency expands.

Confidential & Personal — Prepared for Revanth Acharya | All Systems Operational